ASSIGNMENT-13.5

H.no-2303A51178

Batch – 29

Task 1 – Refactoring – Removing Global Variables

Prompt :

Eliminate unnecessary global variables from the

CODE:

```
Assignment_13.5.py > ...
# rate = 0.1
# def calculate_interest(amount):
# return amount * rate
# print(calculate_interest(1000))
#eliminate unnecessary global variables from the code and ensure that the function is self-contained and does not rely on external state.
def calculate_interest(amount, rate=0.1):
    """Calculates the interest based on the given amount and rate."""
    return amount * rate  # Use the rate parameter instead of a global variable
interest = calculate_interest(1000)  # Call the function with the amount and use the default rate
print(interest)  # Print the calculated interest
```

OUTPUT:

```
100.0
Very Good
```

JUSTIFICATION:

Global variables create tight coupling and make testing difficult. Passing values through function parameters improves modularity, reduces hidden dependencies, and makes functions reusable and predictable.

Task 2 – Refactoring Deeply Nested Conditionals

Prompt :

Refactor deeply nested if–elif–else logic into a cleaner structure.

CODE:

```python
Assignment_13.5.py > ...
# score = 78
# if score >= 90:
# print("Excellent")
# else:
# if score >= 75:
# print("Very Good")
# else:
# if score >= 60:
# print("Good")
# else:
# print("Needs Improvement")
#refactor deeply nested if-elif-else logic into a clean structure
score = 78
if score >= 90:
    print("Excellent")
elif score >= 75:  # Use elif to avoid deep nesting
    print("Very Good")
elif score >= 60:  # Use elif to avoid deep nesting score = 78
    print("Good")
else:
    print("Needs Improvement")  # This will be executed if none of the above conditions are met
```

OUTPUT:

```
Very Good
```

JUSTIFICATION:

Deep nesting decreases readability and increases cognitive complexity. Flattening the logic using guard clauses or mapping improves clarity, simplifies debugging, and enhances maintainability.

Task 3 - Refactoring Repeated File Handling Code

Prompt :

Refactor repeated file open/read/close logic.

CODE:

```python
# f = open("data1.txt")
# print(f.read())
# f.close()
# f = open("data2.txt")
# print(f.read())
# f.close()
#refactor repeated file open/read/close logic.
def read_file(file_name):
    """Reads the content of a file and returns it."""
    with open(file_name, 'r') as f:  # Use a context manager to handle file opening and closing
        return f.read()  # Return the content of the file
print(read_file("data1.txt"))  # Read and print the content of data1.txt
print(read_file("data2.txt"))  # Read and print the content of data2.txt
```

JUSTIFICATION:

Repeated file open–read–close operations violate the DRY principle and increase error risk. Using a reusable function with context managers (with open) ensures automatic resource management, reduces duplication, and improves reliability.

Task 4 - Optimizing Search Logic

Prompt :

Refactor inefficient linear searches using appropriate data structures.

CODE:

```


# users = ["admin", "guest", "editor", "viewer"]
# name = input("Enter username: ")
# found = False
# for u in users:
# if u == name:
# found = True
# print("Access Granted" if found else "Access Denied")
#Refactor inefficient linear searches using appropriate data structurees
users = {"admin", "guest", "editor", "viewer"}  # Use a set for O(1) average time complexity lookups
name = input("Enter username: ")
if name in users:  # Check for membership in the set
    print("Access Granted")
else:
    print("Access Denied")

```

OUTPUT:

```
Enter username: guest
Access Granted
```

JUSTIFICATION:

Linear search requires O(n) time, which becomes inefficient for large datasets. Using sets or dictionaries provides average O(1) lookup time, improving performance and demonstrating appropriate data structure usage.

Task 5 - Refactoring Procedural Code into OOP Design

Prompt :

Use AI to refactor procedural code into a class-based design.

CODE:

```python


# salary = 50000
# tax = salary * 0.2
# net = salary - tax
# print("Net Salary:", net)
#efactor procedural code into a class-based design
class Employee:
    def __init__(self, salary):
        self.salary = salary  # Initialize the salary attribute

    def calculate_net_salary(self):
        tax = self.salary * 0.2  # Calculate tax based on the salary
        net = self.salary - tax  # Calculate net salary by subtracting tax from salary
        return net  # Return the net salary
employee = Employee(50000)  # Create an instance of the Employee class with a salary of 50000
print("Net Salary:", employee.calculate_net_salary())  # Print the net salary by calling the method

```

OUTPUT:

```
Net Salary: 40000.0
```

JUSTIFICATION:

Procedural salary computation lacks structure and scalability. Implementing a class encapsulates related data and behavior, promotes reusability, and aligns with object-oriented design principles.

Task 6 - Refactoring for Performance Optimization)

Prompt :

Use AI to refactor a performance-heavy loop handling large data.

CODE:

```python

# total = 0
# for i in range(1, 1000000):
# if i % 2 == 0:
# total += i
# print(total)
#refactor a performance-heavy loop handling large data
total = sum(i for i in range(1, 1000000) if i % 2 == 0)  # Use a generator expression with sum() for better performance
print(total)  # Print the total of even numbers from 1 to 999999

```

OUTPUT:

```
249999500000
```

JUSTIFICATION:

Iterative loops over large ranges can be computationally expensive. Using mathematical formulas or Python built-in constructs reduces time complexity, improves execution speed, and optimizes performance.

Task 7 - Removing Hidden Side Effects

Prompt :

Refactor code that modifies shared mutable state.

CODE:

```python

# data = []
# def add_item(x):
# data.append(x)
# add_item(10)
# add_item(20)
# print(data)
#Refactor code that modifies shared mutable state.
class DataStore:
    def __init__(self):
        self.data = []  # Initialize an empty list to store data

    def add_item(self, x):
        self.data.append(x)  # Add an item to the data list
data_store = DataStore()  # Create an instance of the DataStore class
data_store.add_item(10)  # Add an item to the data store
data_store.add_item(20)  # Add another item to the data store
print(data_store.data)  # Print the data stored in the data store

```

OUTPUT:

```
[10, 20]
```

JUSTIFICATION:

Functions that modify shared mutable state introduce side effects and unpredictable behavior. Returning values instead of altering globals improves functional purity, testability, and program reliability

Task 8 - Refactoring Complex Input Validation Logic

Prompt :

Use AI to simplify and modularize complex validation rules.

CODE:

```
Assignment_13.5.py > is_valid_password

# password = input("Enter password: ")
# if len(password) >= 8:
# if any(c.isdigit() for c in password):
# if any(c.isupper() for c in password):
# print("Valid Password")
# else:
# print("Must contain uppercase")
# else:
# print("Must contain digit")
# else:
# print("Password too short")

#simplify and modularize complex validation rules
def is_valid_password(password):
    """Checks if the password is valid based on certain criteria."""
    if len(password) < 8:
        return "Password too short"
    if not any(c.isdigit() for c in password):
        return "Must contain digit"
    if not any(c.isupper() for c in password):
        return "Must contain uppercase"
    return "Valid Password"  # Return valid if all conditions are met

password = input("Enter password: ")  # Get password input from the user
print(is_valid_password(password))  # Print the result of the password validation
```

OUTPUT:

JUSTIFICATION:

Nested validation conditions reduce readability and hinder testing. Separating validation rules into individual functions improves clarity, enables independent testing, and supports scalable validation logic.